

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA
ESCUELA PROFESIONAL DE INGENIERÍA DE TELECOMUNICACIONES



TÍTULO:

“Diseño Detallado y Simulación Funcional Completa: Controlador de teclado matricial con banco de registros para ingreso de comandos AT”

INTEGRANTES:

Chuzon Wickham, Renzo Felipe

Rivera Flores, Jorge David

Rodriguez Vasquez, Andrea Alexandra

1. Diagrama Lógico y Esquemático Completo

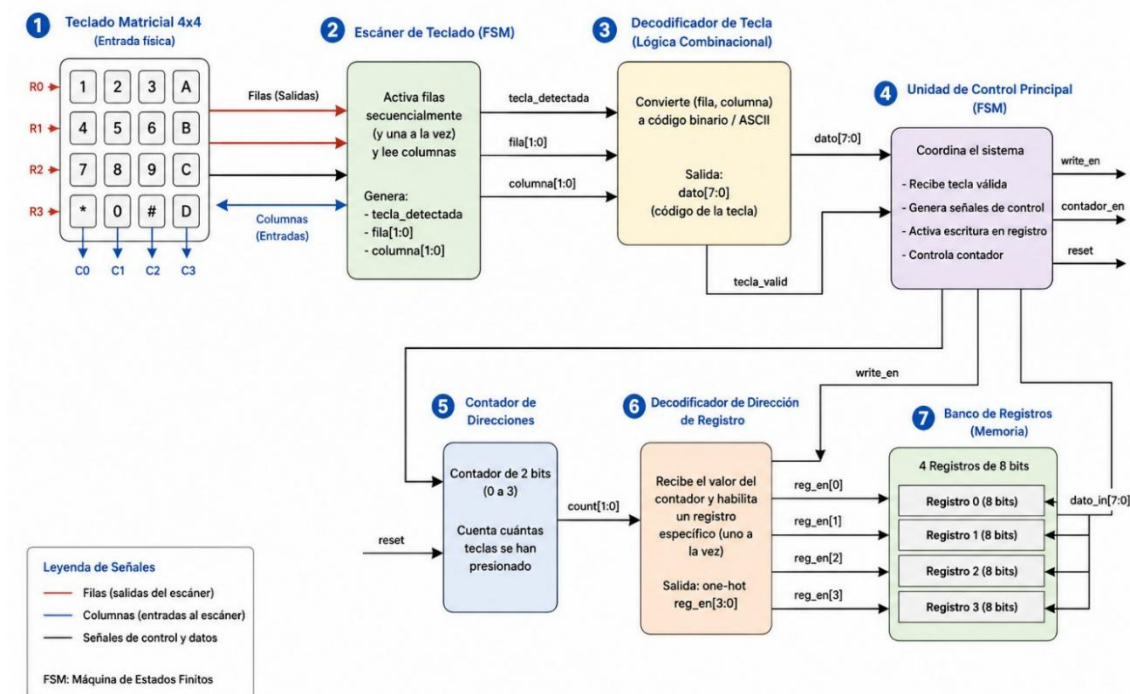


Figura 1.

El diagrama lógico del sistema está estructurado en módulos funcionales que coordinan la lectura física y el almacenamiento digital. La FSM del Escáner de Teclado se encarga de activar secuencialmente las filas (R0 a R3) y leer las columnas (C0 a C3) para detectar una pulsación válida. Una vez detectada, la lógica combinacional del Decodificador de Tecla convierte la coordenada (fila, columna) en su respectivo código binario/ASCII de 8 bits. Simultáneamente, un Contador de Direcciones de 2 bits lleva el registro de la cantidad de teclas presionadas, enviando este valor al Decodificador de Dirección de Registro. Este decodificador combinacional recibe la dirección y habilita exclusivamente uno de los 4 registros del Banco de Registros (Reg 0 al 3), permitiendo que el dato ASCII se almacene en la posición correcta mediante la señal de control general de la Unidad de Control Principal.

2. Diagramas de Transición de Estados (ASM)

La arquitectura del sistema implementa una Máquina de Estados Finitos (FSM) de tipo Moore altamente optimizada, la cual trabaja en conjunto con bloques de lógica secuencial concurrente para gestionar la interfaz entre el usuario y el banco de memoria. Su operación se divide en tres procesos coordinados de forma eficiente:

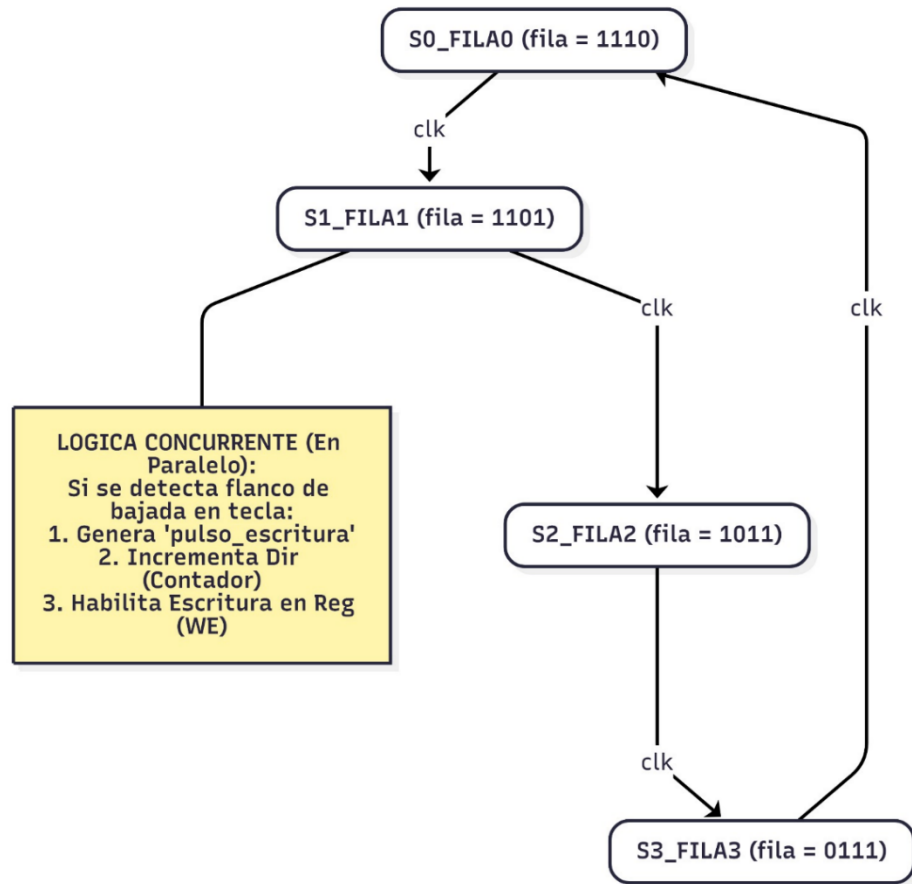


Figura 2.

1. **Fase de Exploración Cíclica (S0 - S3):** En estos estados principales, el controlador realiza un barrido electrónico ininterrumpido de las filas del teclado matricial. Cada estado (S0 a S3) activa una única fila mediante un nivel lógico bajo (0), mientras monitorea el bus de columnas. La FSM transiciona constantemente entre estos cuatro estados en cada ciclo de reloj, garantizando que el sistema esté siempre listo para capturar un evento de entrada sin introducir tiempos muertos.

2. **Detección de Flanco y Control de Pulsación:** Para optimizar los recursos lógicos de la FPGA y evitar la creación de estados de pausa, el sistema prescinde de detener la FSM. En su lugar, implementa un detector de flanco de bajada mediante un registro de desplazamiento básico. Esta lógica concurrente compara el estado actual de la pulsación con el del ciclo de reloj anterior. El resultado es la generación de un único pulso limpio de habilitación justo en el instante en que se presiona la tecla, mitigando las lecturas múltiples continuas causadas por el tiempo que el usuario mantiene presionado el botón.
3. **Ejecución y Almacenamiento Síncrono:** Al generarse el pulso único de habilitación, el sistema ejecuta simultáneamente dos acciones críticas sincronizadas con el reloj maestro, sin necesidad de transitar a un estado de ejecución dedicado:
 - Actualización de Dirección: El contador interno de 2 bits avanza una posición, apuntando a la siguiente dirección de memoria disponible.
 - Escritura en Registro (Write Enable): El decodificador de direcciones enruta la señal de escritura al registro correspondiente (Reg 0 al 3), sincronizando el código ASCII generado por la lógica combinacional con el bus de datos para su almacenamiento permanente.

2.2. Tabla de Verdad: Decodificador de Tecla a ASCII.

Estado ASM	Fila Activa	Columna Leída	Tecla	ASCII (Hex)	D7	D6	D5	D4	D3	D2	D1	D0
S0	Fila 0	Columna 0	1	0x31	0	0	1	1	0	0	0	1
S0	Fila 0	Columna 1	2	0x32	0	0	1	1	0	0	1	0
S0	Fila 0	Columna 2	3	0x33	0	0	1	1	0	0	1	1
S1	Fila 1	Columna 0	4	0x34	0	0	1	1	0	1	0	0
S1	Fila 1	Columna 1	5	0x35	0	0	1	1	0	1	0	1
S1	Fila 1	Columna 2	6	0x36	0	0	1	1	0	1	1	0
S2	Fila 2	Columna 0	7	0x37	0	0	1	1	0	1	1	1
S2	Fila 2	Columna 1	8	0x38	0	0	1	1	1	0	0	0
S2	Fila 2	Columna 2	9	0x39	0	0	1	1	1	0	0	1
S3	Fila 3	Columna 0	*	0x2A	0	0	1	0	1	0	1	0
S3	Fila 3	Columna 1	0	0x30	0	0	1	1	0	0	0	0
S3	Fila 3	Columna 2	#	0x23	0	0	1	0	0	0	1	1
S0	Fila 0	Columna 3	A	0x41	0	1	0	0	0	0	0	1
S1	Fila 1	Columna 3	B	0x42	0	1	0	0	0	0	1	0
S2	Fila 2	Columna 3	C	0x43	0	1	0	0	0	0	1	1
S3	Fila 3	Columna 3	D	0x44	0	1	0	0	0	1	0	0

Esta tabla de verdad detalla la lógica combinacional de decodificación del sistema. En ella se observa la correspondencia entre los estados de escaneo de la FSM (S0-S3), la detección física en las columnas del teclado y la generación del código ASCII en formato de bus paralelo de 8 bits (D7-D0). Esta estructura asegura que cada pulsación sea traducida a un estándar binario compatible con los registros de memoria, permitiendo el almacenamiento de comandos AT de manera estandarizada.

3. Mapa de Memoria y Función del Decodificador

Para el almacenamiento de los comandos ingresados por el teclado matricial, se ha diseñado un banco de memoria de registros de 4 posiciones, cada una con capacidad de 8 bits, totalizando 32 bits de almacenamiento disponible para el comando. Dado que el objetivo es almacenar una secuencia secuencial de pulsaciones, se emplea una arquitectura de direccionamiento lineal controlado por un contador de 2 bits.

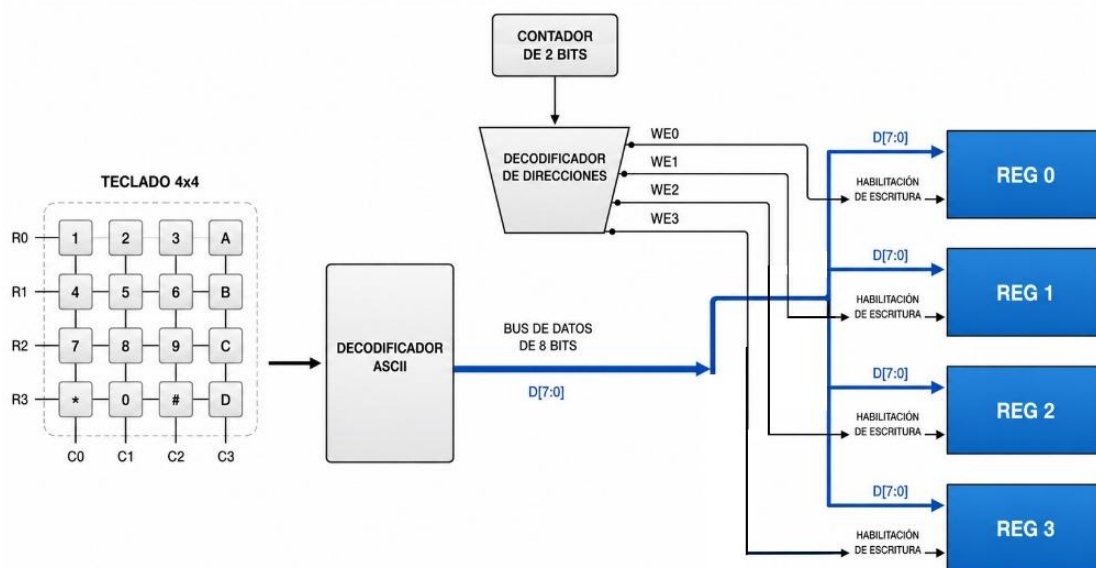


Figura 3.

Tabla 3.1. Mapa de Memoria del Banco de Registros.

Dirección Binaria	Registro	Función en el Comando	Ejemplo de Contenido (Hex)
00	Registro 0 (R0)	Almacena el 1er Carácter ingresado	0x41 ('A')
01	Registro 1 (R1)	Almacena el 2do Carácter ingresado	0x31 ('1')
10	Registro 2 (R2)	Almacena el 3er Carácter ingresado	0x42 ('B')
11	Registro 3 (R3)	Almacena el 4to Carácter ingresado	0x30 ('0')

Para garantizar que cada código ASCII proveniente del decodificador de teclado se guarde secuencialmente en el registro correcto sin sobrescribir los anteriores, se requiere un mecanismo de selección. Esta función es cumplida por el Decodificador de dirección de 2 a 4 líneas.

Este bloque lógico combinacional recibe como entrada la dirección actual generada por un contador de 2 bits que se incrementa cada vez que una tecla es presionada y soltada. El decodificador lee esta dirección binaria y activa una única salida (Write Enable / Clock) que corresponde al registro asociado. Si el contador marca 01, el decodificador activará solo el pin de guardado del Registro 1, manteniendo bloqueados el R0, R2 y R3, preservando sus datos.

4. Código HDL Estructurado (Verilog)

A continuación, se presenta el código Verilog sintetizable para el decodificador y el banco de registros.

```
module controlador_teclado_AT (
    input wire      clk,          // Reloj maestro del sistema
    input wire      reset,        // Reset síncrono activo en alto
    input wire [3:0] col,         // Entradas: Columnas del teclado (Pull-up físico)
    output reg [3:0] row,         // Salidas: Filas del teclado (Escaneo activo en bajo)
    output reg [7:0] reg0,        // Registro de almacenamiento 0 (ASCII)
    output reg [7:0] reg1,        // Registro de almacenamiento 1 (ASCII)
    output reg [7:0] reg2,        // Registro de almacenamiento 2 (ASCII)
    output reg [7:0] reg3,        // Registro de almacenamiento 3 (ASCII)
);

    // 1. MÁQUINA DE ESTADOS (FSM) PARA EL ESCANEO DE FILAS
    // Definición de los estados binarios para el barrido secuencial
    localparam S0_FILA0 = 2'b00;
    localparam S1_FILA1 = 2'b01;
    localparam S2_FILA2 = 2'b10;
    localparam S3_FILA3 = 2'b11;

    reg [1:0] estado_actual;
    reg [1:0] estado_siguiete;

    // Bloque Secuencial: Transición de estados de la FSM
    always @(posedge clk) begin
        if (reset)
            estado_actual <= S0_FILA0;
        else
            estado_actual <= estado_siguiete;
    end

    // Bloque Combinacional: Lógica de cambio de estado y activación de filas (Row)
    always @(*) begin
        case (estado_actual)
            S0_FILA0: begin
                row = 4'b1110; // Activa Fila 0
                estado_siguiete = S1_FILA1;
            end
            S1_FILA1: begin
                row = 4'b1101; // Activa Fila 1
```

```

        estado_siguiente = S2_FILA2;
    end
    S2_FILA2: begin
        row = 4'b1011; // Activa Fila 2
        estado_siguiente = S3_FILA3;
    end
    S3_FILA3: begin
        row = 4'b0111; // Activa Fila 3
        estado_siguiente = S0_FILA0;
    end
    default: begin
        row = 4'b1111;
        estado_siguiente = S0_FILA0;
    end
end
endcase
end

// 2. DECODIFICADOR COMBINACIONAL: TECLA A ASCII (BUS DE DATOS DE 8 BITS)
reg [7:0] ascii_data;
reg       tecla_detectada;

always @(*) begin
    ascii_data = 8'h00;
    tecla_detectada = 1'b0;

    case ({estado_actual, col})
        // FILA 0 (Estado S0)
        {S0_FILA0, 4'b1110}: begin ascii_data = 8'h31; tecla_detectada = 1'b1; end //
        Tecla '1'
        {S0_FILA0, 4'b1101}: begin ascii_data = 8'h32; tecla_detectada = 1'b1; end //
        Tecla '2'
        {S0_FILA0, 4'b1011}: begin ascii_data = 8'h33; tecla_detectada = 1'b1; end //
        Tecla '3'
        {S0_FILA0, 4'b0111}: begin ascii_data = 8'h41; tecla_detectada = 1'b1; end //
        Tecla 'A'

        // FILA 1 (Estado S1)
        {S1_FILA1, 4'b1110}: begin ascii_data = 8'h34; tecla_detectada = 1'b1; end //
        Tecla '4'
        {S1_FILA1, 4'b1101}: begin ascii_data = 8'h35; tecla_detectada = 1'b1; end //
        Tecla '5'
        {S1_FILA1, 4'b1011}: begin ascii_data = 8'h36; tecla_detectada = 1'b1; end //
        Tecla '6'
        {S1_FILA1, 4'b0111}: begin ascii_data = 8'h42; tecla_detectada = 1'b1; end //
        Tecla 'B'

        // FILA 2 (Estado S2) - CORREGIDO (Faltaban el 9 y la C)
        {S2_FILA2, 4'b1110}: begin ascii_data = 8'h37; tecla_detectada = 1'b1; end //
        Tecla '7'
        {S2_FILA2, 4'b1101}: begin ascii_data = 8'h38; tecla_detectada = 1'b1; end //
        Tecla '8'
        {S2_FILA2, 4'b1011}: begin ascii_data = 8'h39; tecla_detectada = 1'b1; end //
        Tecla '9'
        {S2_FILA2, 4'b0111}: begin ascii_data = 8'h43; tecla_detectada = 1'b1; end //
        Tecla 'C'

        // FILA 3 (Estado S3) - CORREGIDO (Faltaban el * y el #)
        {S3_FILA3, 4'b1110}: begin ascii_data = 8'h2A; tecla_detectada = 1'b1; end //
        Tecla '*'
        {S3_FILA3, 4'b1101}: begin ascii_data = 8'h30; tecla_detectada = 1'b1; end //
        Tecla '0'
    end
end

```

```

        {S3_FILA3, 4'b1011}: begin ascii_data = 8'h23; tecla_detectada = 1'b1; end //
Tecla '#'
        {S3_FILA3, 4'b0111}: begin ascii_data = 8'h44; tecla_detectada = 1'b1; end //
Tecla 'D'

        default: begin
            ascii_data = 8'h00;
            tecla_detectada = 1'b0;
        end
    endcase
end

// 3. CONTADOR DE DIRECCIONES Y BANCO DE REGISTROS (CON FILTRO ANTI-REBOTES)
reg    tecla_bloqueada;
reg [2:0] temporizador;
reg [1:0] ptr_direccion;

// El pulso limpio se genera solo si hay tecla detectada Y el candado está abierto
wire pulso_escritura = tecla_detectada & ~tecla_bloqueada;

always @(posedge clk) begin
    if (reset) begin
        reg0 <= 8'h00; reg1 <= 8'h00;
        reg2 <= 8'h00; reg3 <= 8'h00;
        ptr_direccion <= 2'b00;
        tecla_bloqueada <= 1'b0;
        temporizador <= 3'd0;
    end else begin
        // Lógica del Candado con Temporizador (Filtro Anti-Rebote Real)
        if (tecla_detectada) begin
            tecla_bloqueada <= 1'b1;      // Cerramos el candado al detectar la tecla
            temporizador <= 3'd0;        // Reiniciamos el reloj de inactividad
        end else begin
            // Si no vemos la tecla, el temporizador avanza
            if (temporizador < 3'd5) begin
                temporizador <= temporizador + 1;
            end else begin
                // Si pasaron 5 ciclos limpios, recién soltamos el candado
                tecla_bloqueada <= 1'b0;
            end
        end
    end

    // Demultiplexor de escritura en memoria orientada por el contador de 2 bits
    if (pulso_escritura) begin
        case (ptr_direccion)
            2'b00: reg0 <= ascii_data; // WE0 habilitado
            2'b01: reg1 <= ascii_data; // WE1 habilitado
            2'b10: reg2 <= ascii_data; // WE2 habilitado
            2'b11: reg3 <= ascii_data; // WE3 habilitado
        endcase
        ptr_direccion <= ptr_direccion + 1'b1; // Avanza cíclicamente (00 -> 01 -> 10
-> 11)
    end
end
end
endmodule

```

El bloque de hardware descrito cumple estrictamente las reglas de diseño síncrono para síntesis lógica en FPGAs. La separación rigurosa entre bloques combinacionales (para el decodificador

combinacional ASCII y el ruteo de señales de estado siguiente y bloques secuenciales gobernados por flancos de reloj, elimina por completo el riesgo de glitches o la inferencia indeseada de latches lógicos. Asimismo, la asignación por defecto en los bloques combinacionales previene estados indeterminados en el hardware físico.

5. Plan de Simulación

Para validar el funcionamiento del sistema, se ha diseñado el siguiente plan de pruebas empleando un Testbench en ModelSim / Quartus.

6. Capturas de Pantalla Comentadas de la Simulación

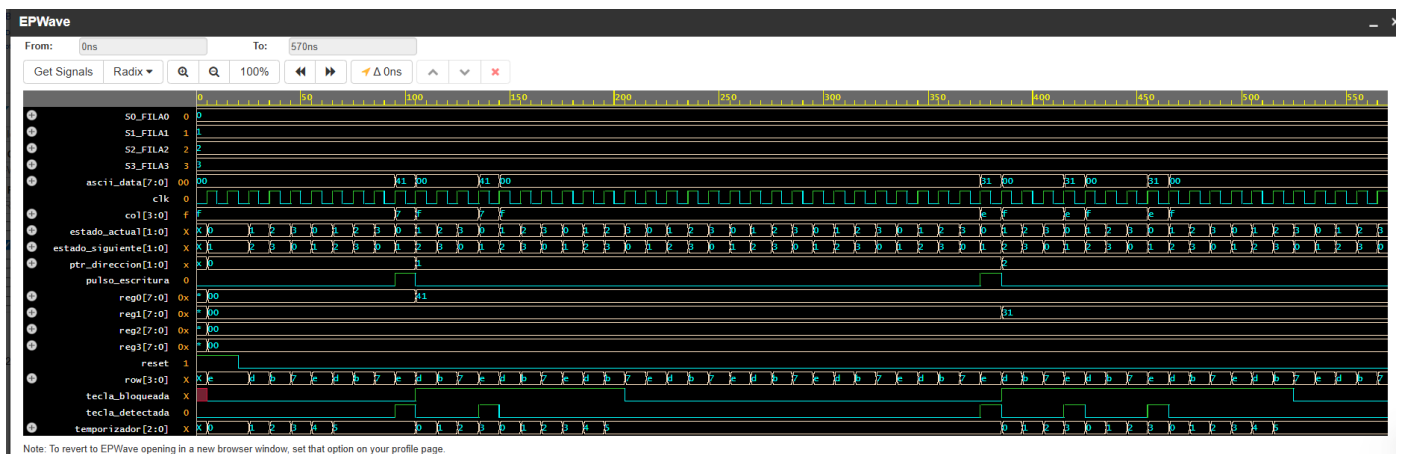


Figura 4. Simulación del código en Eda playground.

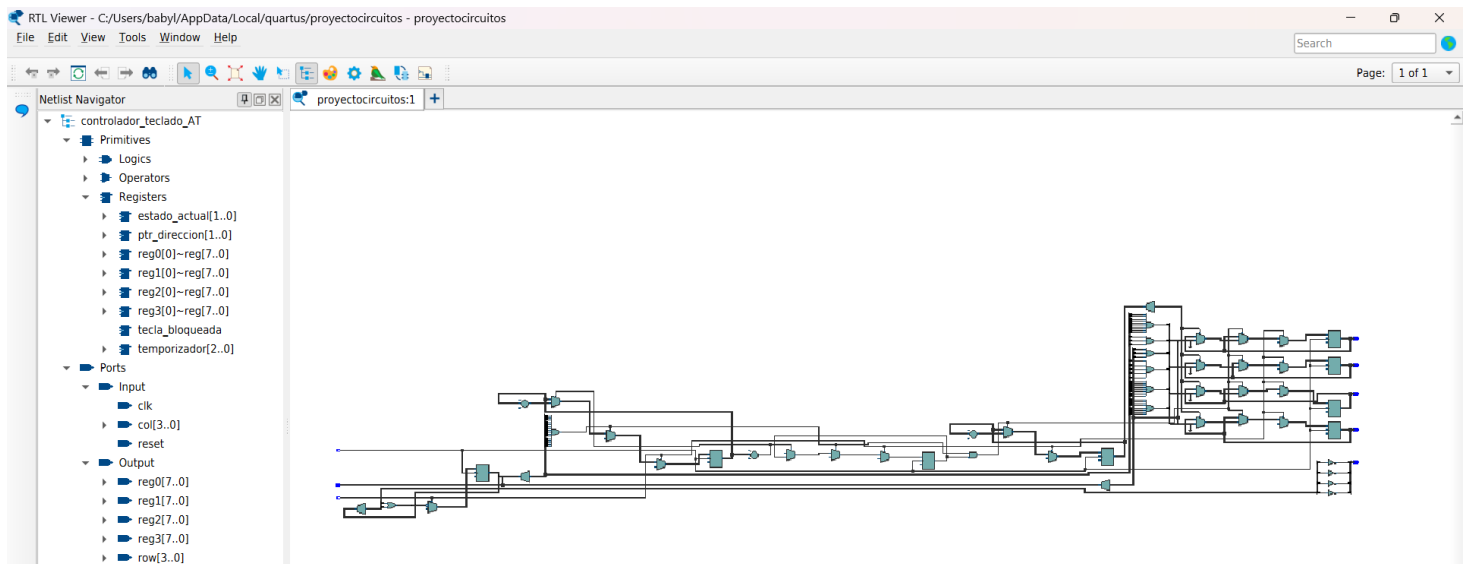


Figura 5. Simulación del hardware en Quartus.

7. Análisis Teórico de las Métricas

- Métrica de Área:

El análisis de área se determina mediante el reporte de síntesis de la herramienta Quartus Prime. En esta métrica se evalúa el consumo de recursos de la familia Cyclone IV E, requiriendo un total de 63 Logic Elements (LEs) y 36 Registros (Flip-Flops) para construir la Máquina de Estados (FSM), el decodificador, el filtro anti-rebotes y el Banco de Memoria de 4x8 bits. Este bajo consumo de compuertas demuestra un diseño altamente optimizado y eficiente en cuanto a espacio físico en el silicio.

- Métrica de Interfaz:

El diseño requiere una métrica de interfaz física de 42 pines lógicos en total para operar: 6 pines de entrada, 1 de reloj, 1 de reset y 4 para las columnas del teclado y 36 pines de salida, 4 para el escaneo de filas y 32 pines dedicados a los 4 registros de 8 bits para la transmisión paralela de los comandos AT

- Métrica de Rendimiento:

El rendimiento del controlador lógico se evalúa en función de su Frecuencia Máxima de Operación (F_{max}). Según el reporte del Timing Analyzer, la lógica interna del diseño es capaz de alcanzar una F_{max} teórica de 367.38 MHz. Sin embargo, debido a las restricciones físicas de conmutación de los pines de entrada/salida, la frecuencia operativa máxima queda restringida a 250.0 MHz. Dado que la interacción humana con el teclado matricial opera en el rango de los milisegundos, este rendimiento demuestra que la arquitectura de hardware propuesta supera con creces las necesidades de la aplicación, garantizando un procesamiento en tiempo real con latencia nula.